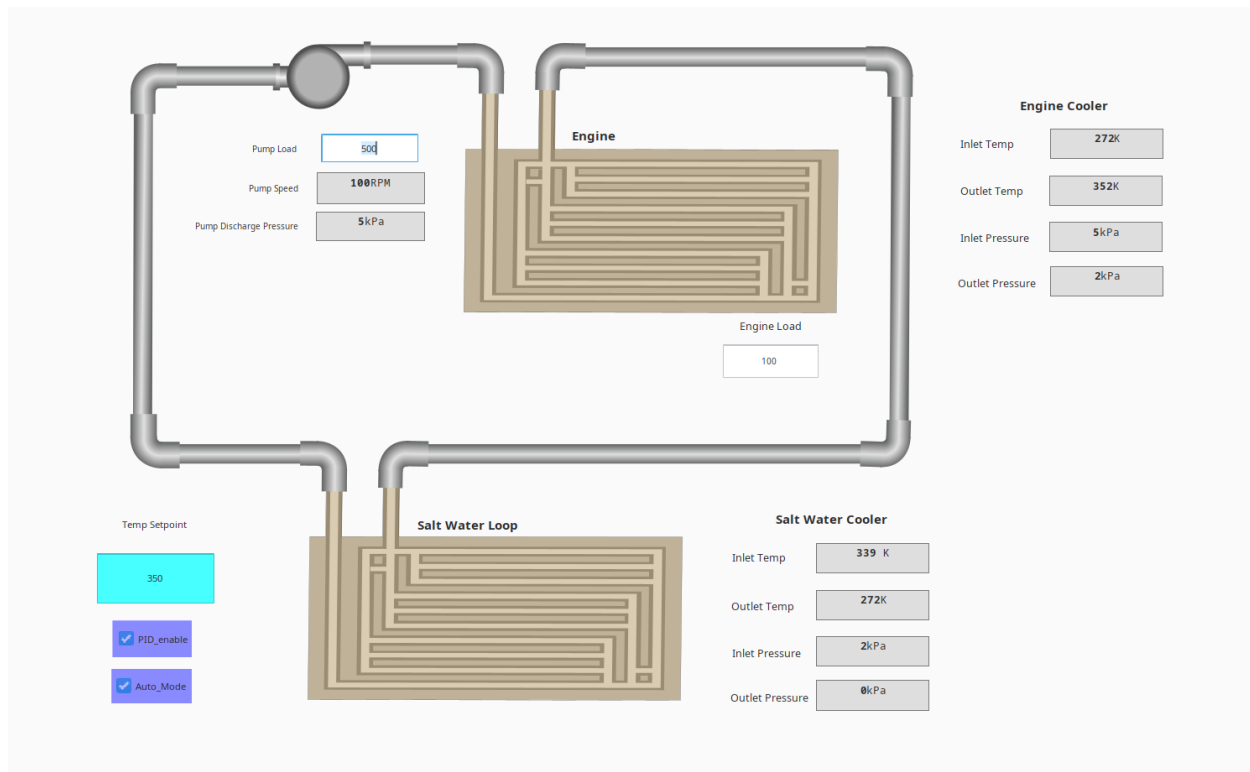


Reactor on Chip (ROC): Simulated Industrial Plant on Raspberry-Pi

Author Name(s): C. R. Parlee; A. M. Moody; and Advisor B. Sarnacki



This project develops a cost-effective, Raspberry Pi-based simulated industrial plant integrated with a SCADA HMI to address gaps in automation education at Maine Maritime Academy. The simulated industrial plant models fluid power, thermodynamics, and Modbus communication using Python. Discrete thermal mass parcels, pump curves, and iterative loss calculations drive a realistic cooling loop. A PLC client reads and writes Modbus registers, enabling closed-loop PID control visualized through an Ignition SCADA HMI.

KEY WORDS: SCADA, HMI, Modbus TCP, Raspberry Pi, Python simulation, PLC, PID control, fluid power, thermal simulation, industrial automation, PyModbus, Ignition, discrete thermal mass model, centrifugal pump, heat exchanger

INTRODUCTION

Industrial automation education is evolving as systems become more interconnected and software focused. At Maine Maritime Academy, the Electronics and Instrumentation Laboratory introduces students to programmable logic controllers (PLCs) and basic automation using tools like Rockwell Automation's Connected Components Workbench. While this builds a solid foundation in control logic, it does not address higher-level systems such as supervisory control and industrial networking.

The current curriculum has limited content involving Supervisory Control and Data Acquisition (SCADA) systems and Human Machine Interfaces (HMIs). Additionally, Connected Components Workbench does not reflect industry-standard HMI practices, which may leave students underprepared for real-world environments. Modern automation relies on fully integrated systems that connect field devices, controllers, networks, and supervisory layers.

This project addresses the lack of a comprehensive platform that allows students to experience complete industrial automation systems. The project proposes developing a cost-effective simulated industrial plant using a Raspberry Pi-based controller integrated with a SCADA HMI. This system will give students hands-on experience across the entire automation stack, helping bridge the gap between theory and real-world application.

BACKGROUND

Industrial automation systems are organized in layers: field devices (sensors and actuators), control systems (PLCs), networking (Modbus, Ethernet/IP), supervisory systems (SCADA/HMI), and enterprise integration. Each layer has its own role in monitoring, controlling, and managing processes.

At Maine Maritime Academy, the Automation and Controls Lab focuses on the field and control layers, so students get limited exposure to networking and supervisory systems—skills that are becoming increasingly important in modern automation.

Due to advancement in software and computing, it is possible to simulate realistic industrial systems at a low cost. Tools like Python and PyModbus make it easy to create and connect virtual devices, while platforms such as Ignition offer professional SCADA and HMI interfaces (Iversen et. al, 2026). The advent of single-board computers allows affordable access to the requisite computational power to support simulations allowing hands-on practice with fully integrated automation environments.

This project expands upon these developments by creating an educational platform that combines simulation, networking, and supervisory control, helping fill the gaps in current automation training.

Contributions to Society

Remedying an Industrial Automation Educational Gap - There is a growing gap between traditional engineering education and modern industry needs. While programs typically cover fundamentals like control theory and PLC programming, they do not fully address the broader scope of industrial automation systems. Students lack exposure to SCADA systems, HMIs, and industrial communication protocols, all of which are key components in real-world operations.

The current curriculum at Maine Maritime Academy focuses on field and control level skills but does not extend to network or supervisory-level systems. As a result, students graduate without a holistic understanding of how industrial systems are integrated, monitored, and optimized.

Industry Demand - Industries such as manufacturing, energy, maritime operations, and infrastructure require engineers with comprehensive automation expertise. Employers seek graduates who go beyond controller programming and demonstrate skills in system integration, data management, and interface design.

Technologies like SCADA platforms, Modbus TCP, and distributed control systems are standard in many industrial plant automation systems. Without experience in these areas, graduates are at a disadvantage, highlighting the need to better align education with industry expectations.

Long-Term Educational Goal - This project aims to create a reusable, scalable digital plant that supports ongoing expansion. Students will develop components—such as control logic, simulated devices, and HMIs—and integrate them into a unified system, encouraging hands-on and collaborative learning. Over time, successive student groups can build on previous work, resulting in a progressively more advanced simulated plant while reinforcing system integration.

ENGINEERING TRADEOFFS

Microcontrollers were first considered for use in this project given their programming simplicity, minimal software bloat, and low cost. The ESP32 was the first microcontroller selected given its integration into the Arduino ecosystem, decent clock speed, and the authors' familiarity with the device. However, the Automation Lab SCADA network uses EtherNet with RJ45 connectors. ESP32 development boards do exist with RJ45 terminals for use in EtherNet networking, shown in Figure 1, but they are uncommon, and their cost reflects that.



FIGURE 1: ESP32-S3 Ethernet development board (Amazon).

Due to the increased cost of Ethernet-supported hardware, the decision was made to move the project to a single-board computer (SBC) rather than a set of microcontrollers.

The Raspberry Pi 5 was selected due to its 2.4 GHz clock speed while the ESP-32 only has a 240 MHz clock. Although the ESP-32 has less operating overhead, the Raspberry Pi is significantly more scalable. The additional computing power would allow the Raspberry Pi to host several processes that each simulate a plant rather than several microcontrollers simulating a plant each, making the Raspberry Pi more scalable and cost-effective. The cost tradeoff study is shown in Table 1 and contains prices from late 2025. The pricing comparison assumes that the Raspberry Pi can simultaneously simulate eight plants for the eight stations in the Automation Lab.

TABLE 1. Cost Comparison Between ESP32 Ethernet vs. Raspberry Pi 5 SBC for Plant Simulation

Part	QTY	Cost	Subtotal
Raspberry Pi 5 (8 GB)	1	\$ 85.00	-
SD Card	1	\$ 8.80	\$ 93.80
ESP-32 Ethernet	8	\$ 16.60	\$ 132.80

Hence, Table 1 shows that while the single-part cost of the Raspberry Pi 5 is more than the ESP-32, the scalability offered by the Raspberry Pi allowed it to be more cost-effective than the ESP-32.

SIMULATION DESIGN

The project simulation is a base-principles fluid power and thermodynamic simulation run in Python 3.12.3. The additional required dependencies and their versions can be found in Appendix A1 or on the online code repository for this project (Parlee). To maximize codebase extensibility, the key systems are modular and comprise fluid power/flow, Modbus, and thermal mass transport.

Fluid Power and Devices

The fluid power logic of the system is handled by a naming scheme analogous to finite element analysis. All of the “physical” components in the system are called elements, or devices. This includes pipes, valves, pumps, heat exchangers, etc. These elements are connected to one another using nodes. This means that the outlet of a pump is a node that is the inlet of a pipe. A visualization of this is shown in Figure 2.

It shall be noted that custom classes shall be denoted in Pascal case, for example, `MyClass`. Functions will be denoted by their plain text name, italicized, for example, `functionExample()`.

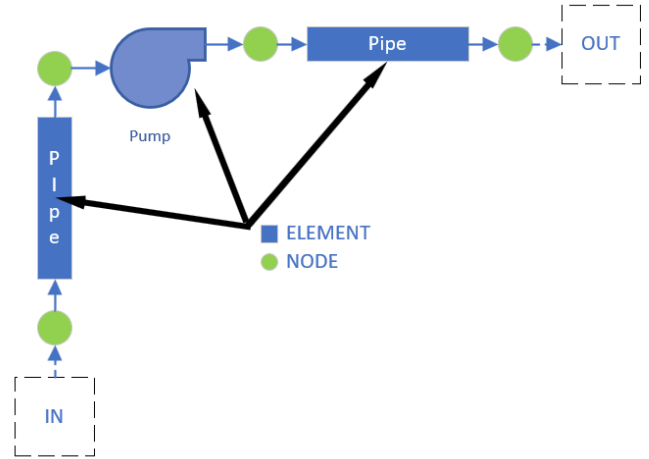


FIGURE 2. Demonstration of nodes and elements within simulation context.

The nodes of a system are handled by a container class called `NodeManager`. This should be declared early in program execution as all devices require a `NodeManager` object as a parameter to get their registered node objects. The instantiation of a `NodeManager` object has no required parameters but accepts a default material for the fluid contained within.

`NodeManager` contains a function `addNode()` which has an optional argument of a node. This will add the node to the list of stored nodes within the `NodeManager` and if the node does not exist, the node is created.

The `NodeManager` class contains properties that define default temperature, pressure, and mass flow units. Pressure and flow are handled by the `ControlLoop` class, and temperature is handled by the `FluidParcelManager` class.

Device Polymorphism

In the design of the device architecture, it was known that all devices needed a few methods while some devices made modifications to those methods or added new behaviors entirely. Hence, it was decided that the object-oriented programming approach of polymorphism would be used. The base class is the

Device class, which simply contains a list of Modbus register addresses that are related to that device. Both DeviceInline and DeviceParallel inherit from this Device class.

DeviceInline is the class for devices that fluid will flow through. This includes pumps, heat exchangers, pipes, and valves. The DeviceInline constructor accepts additional optional methods for the inlet and outlet nodes (which will be created if they are omitted), a k minor-loss coefficient, diameter, volume, and fluid path length. The DeviceInline class contains functions which handle device-level calculations for pressure drop from minor losses. DevicePipe is the only class which handles major loss calculations in addition to minor losses.

DeviceParallel inherits from the Device class and is designed to work with components that are attached to a flow path but don't necessarily receive flow themselves. This is generally limited to most sensors (i.e., temperature and pressure). To instantiate a DeviceParallel, an attached node must be provided.

An inheritance tree of most of the device classes are shown in Figure 3.

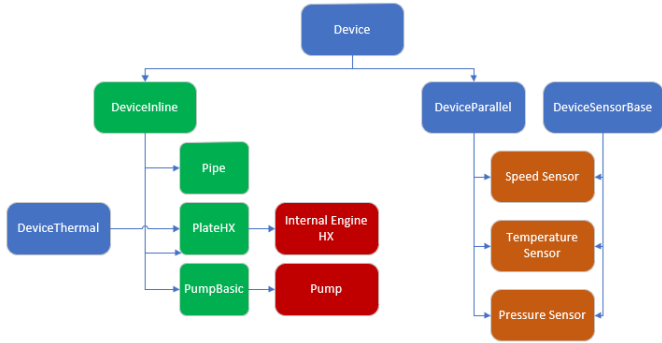


FIGURE 3: Inheritance tree of simulation devices.

Pumps

The pump is a special implementation of the DeviceInline class. Described in the Pressure and Flow section, the Pump class creates pressure and flow given a pump curve. In this case, the curve is based upon a Grundfos TP 40-240/2 A-G-A-BQQE-GX2 inline centrifugal pump. The pump curve is converted into a second-order polynomial and provided to the pump using the `setPumpCurve()` method. The pump in the created cooling loop simulation uses the pump curve

$$H = -241374 \cdot q^2 - 2.2726 \cdot q + 19.105 \quad (1)$$

where q is the system flow rate in cubic meters per second. The pump curve is scaled in q and H by the process point value, which was initially set directly by the `set` function. However, to model a pump more accurately, the process point is changed over time to “ramp” up the pump when it changes speed. This is modelled by the function

$$g_P = g_{set} + (g_{last} - g_{set}) \cdot e^{-\frac{\Delta t}{\tau}} \quad (2)$$

where g_{set} is the desired setpoint, g_{last} is the last process point before the system changed, Δt is the time since the change, and τ is the pump time constant. This is shown in Figure 4.

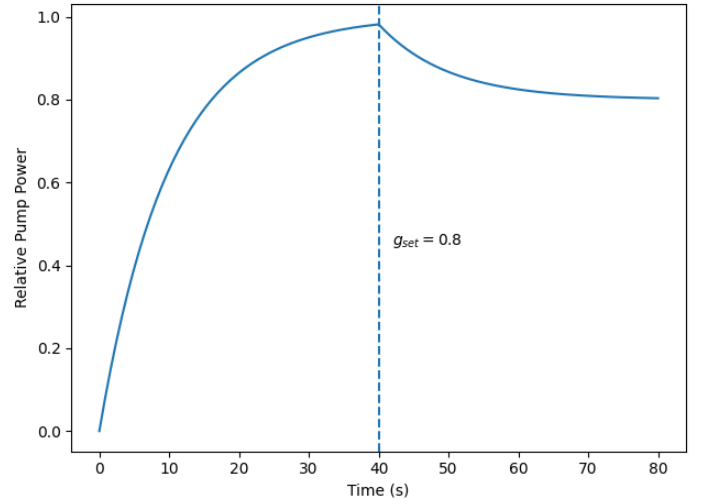


FIGURE 4: Pump ramp and setpoint change.

The pump time constant is set to 10 seconds to show ramp effects.

Pressure and Flow

Within the DeviceInline classes, `computeMinorLoss()` can be used to determine the minor losses in the element for a provided flow parameter in cubic meters per second. Minor loss is computed by first determining the fluid velocity, v :

$$v = \frac{\dot{V}}{A} \quad (3)$$

Where A is the predefined area property for the element and $\dot{V}$ is the flow property. Next, minor loss is determined in meters of head as:

$$H_m = k * \frac{v^2}{2g} \quad (4)$$

Where g is the acceleration of fluid due to gravity and k is the minor loss coefficient for the element, defined statically.

Major losses are computed for all instances of DevicePipe. These devices are stored within the pipes variable of the ControlLoop instance. For a given flow rate, $\dot{V}$, a total head loss required is calculated iteratively as follows:

$$H = \sum_{i=0}^n H_i(\dot{V}) \quad (5)$$

where H_i is the head loss of the pipe as calculated by the *computeMajorLoss()* function. This function computes fluid velocity, v , as is calculated in Eq. 3. Using fluid velocity, Reynolds number is calculated as

$$Re = \frac{\rho v D}{\mu} \quad (6)$$

where ρ is the density of the fluid medium, D is the diameter of the pipe, and μ is the viscosity of the fluid. Reynold's number is then used in the Swamee-Jain equation to determine friction factor:

$$f = \frac{0.25}{\left[\log\left(\frac{\epsilon}{3.7D} + \frac{5.74}{Re^{0.9}}\right) \right]^2} \quad (7)$$

with ϵ representing roughness. However, if Reynold's number is zero, the equation collapses to simply:

$$f = \frac{0.25}{\left[\log\left(\frac{\epsilon}{3.7D}\right) \right]^2} \quad (8)$$

To remove the singularity which occurs at $Re = 0$. Major loss is finally calculated as

$$H_{mj} = f \cdot \frac{l}{D} \cdot \frac{v^2}{2g} \quad (9)$$

A graph of these major and minor losses are shown in Figure 5.

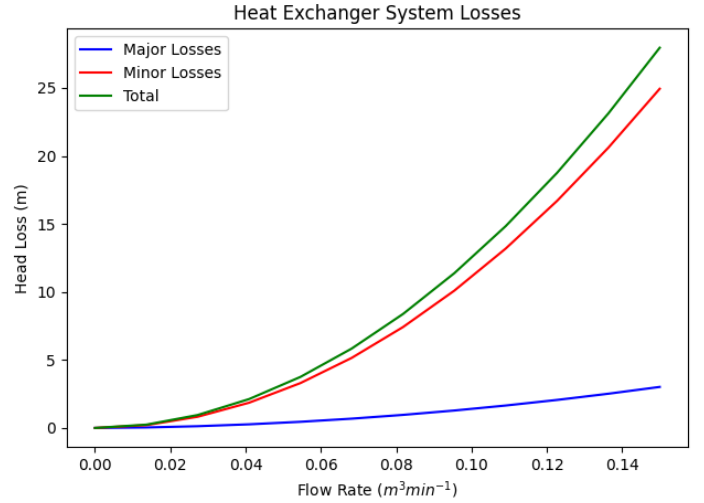


FIGURE 5: Plot of major, minor, and total losses for the as-designed two heat exchanger system.

To determine the system operating point, the pump curve is used to find the point of intersection between the total loss curve and the pump curve. This behavior is shown in Figure 6.

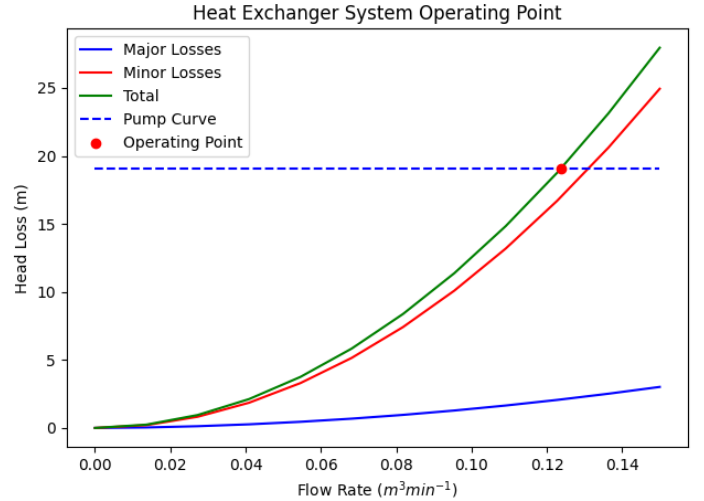


FIGURE 6: Plot of heat exchanger operating point from total loss and pump curves.

This operating point is determined by the *computeOpPoint()* function within the ControlLoop class. The Scipy library's Brentq root finding algorithm is used on a lambda equation defined as

$$f(\dot{V}) - H_{mi}(\dot{V}) - H_{mj}(\dot{V}) \quad (10)$$

where f is the pump curve and H_{mi} and H_{mj} are the minor and major loss functions, respectively. This equation will approach zero at the point of intersection. Hence, a point is returned from the *computeOpPoint()* function that corresponds to $(\dot{V}, H(\dot{V}))$.

MODBUS

The primary element of the Modbus system is the `ModbusManager` class. This class, once instantiated, records references to a list of sensor objects using the `addSensors()` method. The `addSensors()` method is agnostic to specific subclasses of device but that class must have both a `get()` and `getScale()` method for reporting the value of the sensor, as explained later.

The current implementation of the `ModbusManager` class initiates 100 discrete inputs (read only), coils (read/write), holding registers (read/write 16-bits), and input registers (read only 16-bits). Currently, only the holding registers and input registers are used, as it was not deemed necessary to introduce coils or discrete inputs for the purpose of this simulation. The starting address for each of these communication types is shown in Table 2.

TABLE 2. Starting Register Addresses for All Data Types

Type	Data (read/write)	Starting Address	Range
Discrete Inputs	On/off (r)	100	100-199
Coils	On/off (r/w)	200	200-299
Input Registers	16-bit (r)	300	300-399
Holding Registers	16-bit (r/w)	400	400-499

The `ModbusManager` class is designed to run on an asynchronous thread. In this instance, `Asyncio` is used as it adds additional features beyond the `Threading` class (“`Asyncio` — Asynchronous I/O”; “`Threading` — Thread-Based Parallelism”). A wrapper function called `start_server()` is created which calls `asyncio.run()` containing the `ModbusManager` `run_server()` method.

The `run_server()` method accepts two optional arguments: an address of an adapter to broadcast on and a port. The two suggested addresses are 127.0.0.1, which broadcasts within the device’s subnet, and 0.0.0.0, which broadcasts to all adapters. 127.0.0.1 is the default value. The default port is 5020.

To handle running multiple virtual plants, each plant would be localized to different ports on the same address. For instance, lab station 1 might be broadcast to 0.0.0.0:5021, meaning all adapters, port 5021.

Once the `start_server()` function has been defined, a thread is created using `threading.Thread()` with the target parameter selected as the `start_server()` method. The thread is started with the `start()` method.

The benefit of using an asynchronous thread for the `ModbusManager` thread is that the device can respond to calls from communicating devices without the danger of being held up by synchronous operations being run on another thread. If these operations were taking place synchronously and a timely

computation, such as total system flow, was also taking place, the Modbus database would be unable to respond to requests for information. There are ways around this by using interrupts, but for extensibility and ease of use, it is considered best practice to use asynchronous calls.

Holding Register Callbacks

Rather than continuously updating values tied to holding registers which may be updated outside of the program, the `ModbusManager` monitors the data values in the holding register context and can trigger functions based upon changes that take place at certain addresses. This is done by calling the `register_hr_callback()` method at execution time. This function accepts arguments of address and callback, where address is the address to monitor for changes and callback is the method or function to call if a change occurs.

The asynchronous function `monitorHoldingRegisters()` should be run in a new asynchronous thread so that it can trigger callbacks without being held up by executions in other threads.

When a monitored address is changed, the callback function is called with three parameters: the address that triggered the callback, the old value, and the new value.

Modbus Server Troubleshooting

It became necessary on many occasions in the development of this simulation to troubleshoot the data from the Modbus server. `Modbus Tool` by fesse was the first utility used (“`Modbus Tool`”). Over time, the project transitioned to `OpenModscan` due to its compatibility with Linux (Ananьев, 2026).

The use of these tools allowed for the troubleshooting of the problems involving Modbus-facing values that emerged as the server transitioned from the Windows-based development server to the final Raspberry Pi production environment.

Issues in this development primarily manifested in two instances: improper adapter selection and 1-based indexing. When these tools were used in the development environment, Modbus communication was taking place within the local host. Hence, the selection of the server address as the local host/loopback address, 127.0.0.1, did not cause communication issues. However, once the server was moved off the local host, the loopback address did not allow for communication to occur from the client to the gateway and then to the server and back.

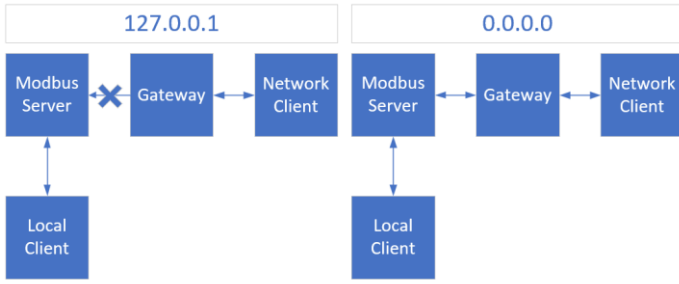


FIGURE 7. Representation of server communication via loopback vs all adapters

Challenges also emerged from the zero-based indexing schema present in Python. By default, address 300, for example, is stored in an address corresponding to 300. However, when interfacing with address 300 from a PLC, the default behavior is one-based indexing. The PLC then translates the address into the default zero-based indexing. If a user is writing code with this zero-based schema in mind, the address targeted by the code will be one offset from the true address. Hence, requesting address 300 from a PLC will be translated into address 299. This will only result in errors because address 299 will be undefined for that function code. To request a value, a function code (i.e. for a discrete input or holding register) must be provided in addition to the address. This offset address will result in error 0x02, illegal data address. Once the listening address was changed to 0.0.0.0, communication was opened in the production environment to the other devices on the network.

The next challenge arose from Modbus indexing. Pymodbus (and the Modbus protocol) uses zero-based indexing, meaning that the address 400, for example, is stored at the zeroth value and isn't offset. PLCs, however, often use one-based indexing (Ananev) (Socomec). So, to access address 400 on a PLC, the PLC program must request address 401 despite using the same protocol.

THERMAL MASS TRANSPORT

Once the framework for fluid power was built, it became necessary to decide how system temperatures for both fluid and devices would be modeled. The original model considered a lumped-parameter thermal network model. This meant that, like the fluid power model, the system is broken into nodes which each have a capacitance, i.e. a resistance to temperature change. The heat flux would be driven by temperature differences and mass flow rate of fluid in the system. However, several issues were anticipated with this model.

First, the model would not describe transportation delays in the system if the loop were long. This means that hot fluid from a heat exchanger might instantly be added to the cooler unless some artificial delays were added. These delays would not make the overall model easier to create than alternative modeling

techniques, as they would require a queue for when temperature change should occur in the consecutive node. This could be computationally very costly depending on the number of nodes in the system and the length of the delay. This also means that the system could not effectively model transients as they require time delays to arrive at a steady state. In essence, the system would always be at steady state. The final reason for abandoning this technique was the need for root finding to determine instantaneous temperature change through a heat exchanger. The following equations can be used for temperature change in a heat exchanger

$$q = \dot{m}c_p\Delta T \quad (11)$$

where q is the rate of heat transfer through the system, $\dot{m}$ is the mass flow rate of the fluid through the heat exchanger, c_p is the specific heat of the heat exchanger, and ΔT is the temperature difference between the inlet and outlet of the heat exchanger. The other equation is

$$q = UA\Delta T_{LM} \quad (12)$$

Where U is the overall heat transfer coefficient, A is the surface area of the heat exchanger, and ΔT_{LM} is the log-mean temperature difference of the fluid entering and exiting the heat exchanger.

However, at no point does the simulation know both the inlet and outlet temperatures of the exchanger, hence, both equations must be solved simultaneously with a root-finding algorithm to determine the outlet temperature from the inlet or vice-versa. From there, q can be determined. However, this simulation must be able to run as close to real time as possible, making the use of multiple minimization algorithms for temperature simulation unfeasible.

Discrete Thermal Mass Model

Given the shortcomings of the lumped-parameter thermal network model, it was decided that a discrete thermal mass model would be used. This model comprises a system of discrete mass "parcels" evenly distributed throughout the system. The parcels travel through the system at speeds driven by the system flow rate and transfer heat to the element that they are in.

Mass parcels are spawned by the FluidParcelManager class, which accepts a reference to a ControlLoop and has an optional parameter, n , for how many parcels it should create with a default of 20. When instantiated, the FluidParcelManager class determines the total system volume from the ControlLoop using the equation

$$V = \frac{\pi D^2 L}{4} \quad (13)$$

for cylindrical devices, where D is the diameter of the pipe and L is the length. However, some devices may have explicitly defined volumes (such as heat exchangers), and these are added directly to the sum of the volumes.

Using the total system volume, the system fluid mass is determined using

$$m = \rho \cdot V \quad (14)$$

where ρ is the density of the fluid (water, in this case, and assumed to be constant and not affected by temperature or pressure). From there, the differential mass for each parcel is determined as

$$dm = \frac{\sum m_i}{n} \quad (15)$$

where m_i is the mass of each individual element and n is the total number of parcels to be distributed.

Next, the total length of the flow path is determined by summing each of the elemental length values. Then, the position of each parcel is determined by

$$x_i = \frac{L}{n} i \quad (16)$$

This position is then provided to the *determinePositionFromLen()* function, which returns a corresponding element and fractional position (between 0 and 1) for the parcel. From these calculated elements, position and mass, a parcel is finally created. The parcel is also assigned with an initial temperature.

After initialization, parcels are checked every cycle and their positions are updated. Their velocities are first determined using equation 3. From there, a change in fractional position is determined using

$$dp = \frac{dt \cdot v}{l} \quad (17)$$

where dt is the time since the last cycle execution and l is the length of the containing element. This is added to the parcel's fractional position. When this state reaches or exceeds 1, the parcel is added to the following element and removed from the current element.

Thermal Devices

Thermal energy transfer occurs when the parcels are within an element which descends from *DeviceThermal*. A ΔT is determined in the *FluidParcelManager* class between the parcel

and the element. Using this temperature difference, heat flow is calculated from the thermal mass using the *DeviceThermal* function *getHeatFlow()*. This function requires arguments of area, temperature differential, and time. Area is calculated as a fraction of total element heat flow area using

$$A = A_t \cdot \frac{m_p}{m_t} \quad (18)$$

where A_t is the total heat exchanger area and m_p and m_t are the individual parcel mass and total mass within the heat exchanger, respectively. This effectively calculates the fraction of total heat exchange surface area that each parcel of mass is in contact with at any given time, allowing for varied heat flow.

The *getHeatFlow()* function assumes that the heat flow within an element can be modelled as a lumped capacitance system. This means that convection and conduction are treated as series resistances to heat flow. When thermal devices are instantiated, conductive resistance is calculated using

$$r_{cond} = \frac{l_c}{k \cdot A} \quad (19)$$

where l_c is the characteristic length, often the thickness of the heat exchanger plate, and k is the thermal conductivity. l_c is set as 0.1 meter and k is set as 50 W/m-K for both heat exchangers modelled in this simulation.

Convective resistance is calculated with

$$r_{conv} = \frac{1}{hA} \quad (20)$$

where h is the convective heat transfer coefficient in W/m²-K. In this model, to alleviate the need to calculate Nusselt and Prandtl numbers, h is defined arbitrarily as 3,000 W/m²-K.

Using these constants, the *getHeatFlow()* function returns a heat flow using

$$Q = \frac{\Delta T \cdot A \cdot \Delta t}{R_T} \quad (21)$$

where R_T is the total resistances of the r_{conv} and r_{cond} for that element.

The energy determined from this function is then removed from the thermal device, which is treated as a bulk thermal mass. Temperature changes for the thermal device and parcel are calculated using

$$\Delta T = \frac{Q \cdot \Delta t}{m \cdot c_p} \quad (21)$$

with Q being the energy transferred, Δt being the time change, and C_p is the specific heat of the mass.

SHORTCOMINGS

The shortcomings of this model include the assumption that mass is evenly distributed throughout the mass parcels and that it is based upon the effective length of the components of the system. This is only a reasonable assumption when the volume of fluid contained per length within each component is the same. This means that when distributing fluid, a small heat exchanger with thin tubes will have the same amount of fluid in a given space as a relatively large pipe. This is not reasonable but does not appear to affect the accuracy of the simulation, and this phenomenon would be greatly decreased as the number of fluid parcels in the system is increased.

Additionally, the arbitrary use of a constant convective heat transfer coefficient does not accurately model the thermal resistance's dependence on flow rate. Flow rate drives the convective heat transfer coefficient through the Nusselt number, which is driven by Reynold's number, a function of fluid velocity.

Building from the Framework

With the simulation tools built, a simple cooling loop can be built from the tools themselves. Pipe diameter is set to 1.5 inches (0.0381 meters), pipe roughness is 4.6E-5 meters, density is 999 kg/m³, and viscosity is 1.02E-3 m²/s. A dynamic pump is used (slow ramp up, ramp down) and two coolers are used. The hot-side cooler slowly adds heat to the heat exchanger thermal mass based upon an artificial engine load. The cold side heat exchanger rejects heat to a constant temperature medium that is 293°K. Three pipes are in the system with varying lengths from 5-10 meters. There are pressure sensors at the pump, cooler and heater inlet and outlet, and a speed sensor on the pump. The heater and cooler have inlet and outlet temperature sensors. A model of the system is shown in Figure 8 with a sheet of addresses found in APPENDIX A2.

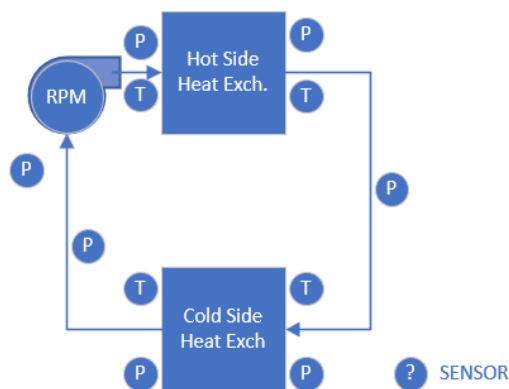


FIGURE 8. Flow chart model of simulated system

SOFTWARE TOOLS USED

The following software tools were utilized in the development of this system. Connected Components Workbench (CCW) was used for programming the PLC, configuring communication settings, and defining system variables. It also provided built-in tools for testing and monitoring the control logic during development, ensuring proper functionality before deployment.

The Ignition Designer Launcher was used to develop the Human-Machine Interface (HMI). Within this environment, PLC variables were imported as tags, enabling real-time process data to be displayed and controlled. Graphical elements such as indicators, buttons, and input fields were implemented to create an interactive and user-friendly interface.

PLC AND MODBUS INTEGRATION

The initial phase of this portion of the project involved configuring the programmable logic controller (PLC) and establishing communication with the Modbus-based simulation environment. This required developing an understanding of how Connected Components Workbench manages communication parameters, including device addressing, register mapping, and protocol configuration. The PLC was configured as a Modbus client, allowing it to exchange data with the Raspberry Pi-based Modbus server. Proper configuration of communication settings such as IP address, port, and node identification was essential to ensure reliable data transfer between devices.

A key component of this implementation was the use of the MSG_MODBUS2 instruction block within the PLC program. This block facilitates communication between the PLC and external Modbus devices by allowing the reading and writing of register values. When configured for reading operations, the block retrieves data from input or holding registers on the target device and stores the values in designated PLC memory locations. For writing operations, the block takes data from the PLC and transmits it to specified holding registers on the Modbus server. This bidirectional functionality enables real-time data exchange between the control system and the simulation environment.

The MSG_MODBUS2 block requires both local and target configuration parameters. Local configuration defines where and how the data is stored within the PLC. The parameters include the starting register and the number of elements to be read or written. Target configuration specifies the Modbus server details, including the device address, register type, and starting register on the remote system. Ensuring the configuration of these parameters is crucial for proper communication.

A primary challenge encountered while establishing communication from the PLC to the Modbus was managing Modbus register indexing. Addressing conventions can differ depending on whether the system uses zero-based or one-based indexing. This resulted in an increase or decrease of register addresses by one to correctly align the PLC data with the intended Modbus register. This adjustment was particularly important

when switching between reading and writing operations, as inconsistencies in indexing could result in incorrect data being accessed or overwritten.

Data Scaling and HMI Integration

Once communication between the PLC and Modbus server was established, the next step involved processing and scaling the incoming register data. The values retrieved from Modbus registers are transmitted as raw 16-bit integers which then needed to be converted to other data types to allow the PLC to use the data. The various temperature and pressure values were scaled on the Modbus end. Therefore, scaling was implemented within the PLC logic to convert these raw values to accurately reflect the simulated process conditions.

In Connected Components Workbench, variables must be defined as global to be accessible within the HMI design environment. As a result, all process variables intended for monitoring or control were declared as global variables within the PLC program. These variables were then mapped as tags within the HMI configuration, allowing them to be displayed and manipulated through the user interface.

While this approach ensured functionality, it also highlighted the importance of variable management. In hindsight, a more structured approach using a combination of local and global variables would improve code organization and maintainability. Restricting global variables to only those required for HMI interaction would reduce unnecessary memory usage and improve overall clarity of the control program.

The final HMI is shown in Figure 9. It displays the PID controller enabled with auto mode turned on.

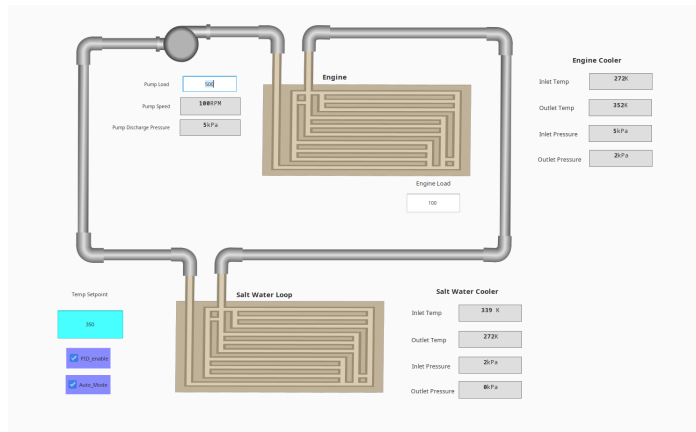


FIGURE 9. Final HMI

PID Implementation and Tuning

To improve system realism and functionality, a proportional–integral–derivative (PID) control loop was added to the PLC program. This enabled closed-loop control, allowing the system to automatically adjust process conditions based on feedback.

The operator could switch between manual and automatic modes through the HMI.

The PID block was integrated with the HMI using global variables mapped as tags. A “PID Enable” checkbox controls whether the loop is active. This must remain enabled for both modes. An “Auto/Manual Mode” selection determined system operation. In manual mode, the operator directly controls pump speed and engine load. In automatic mode, the operator adjusts the setpoint (engine outlet temperature), and the PID controller adjusts the pump speed to maintain this value.

The PID loop uses the engine outlet temperature as the process variable and pump speed as the control variable. The setpoint was entered through the HMI. Tuning was performed at 100% engine load to simplify system response. To more accurately tune this system, several bump tests would need to be made at various engine load values. It was found that the system acted more linear in the higher engine load ranges. A bump test was used to characterize system behavior by introducing a step change in the input and monitoring the resulting output response. From this test, the PID gains were estimated.

The PID controller block uses three tuning parameters—proportional gain (K_c), integral time (T_i), and derivative time (T_d)—to regulate the process by minimizing the error between the setpoint and the control variable.

The proportional gain (K_c) determines the controller’s response to the current error. Increasing K_c results in a stronger and faster reaction to deviations from the setpoint, but excessively high values can lead to instability and oscillations.

The integral time (T_i) controls the accumulation of past error. The integral term eliminates steady-state offset by integrating the error over time. Smaller values of T_i increase the influence of the integral action, improving accuracy but potentially causing overshoot.

The derivative time (T_d) predicts future error based on the rate of change of the process variable. This action helps dampen the system response and reduce overshoot. However, derivative action is sensitive to noise and is often minimized or omitted in practice.

The PID control law in standard (ideal) form is given by:

$$u(t) = K_c \left[e(t) + \frac{1}{T_i} \int_0^t e(\tau) d\tau + T_d \frac{de(t)}{dt} \right]$$

where:

$u(t)$ = controller output

$e(t)$ = error (setpoint – process variable)

K_c = proportional gain

T_i = integral time

T_d = derivative time

Improper tuning of a PID controller can lead to severe performance issues that may result in equipment damage or failure. Excessive proportional gain or insufficient damping can cause sustained oscillations, placing mechanical stress on actuators such as pumps and valves. Similarly, overly aggressive integral action may introduce significant overshoot, potentially driving the system beyond safe operating limits for temperature, pressure, or flow. In extreme cases, these conditions can accelerate wear, trigger protective shutdowns, or cause catastrophic failure of system components. Therefore, careful and systematic tuning is essential to ensure stable operation and to maintain the safety and longevity of the equipment.

Final tuning parameters were $K_c = 5$, $T_I = 15$, and $T_D = 0$, with control output limits between 0 and 1000.

The reverse acting process required a direct acting controller. Therefore the “Control” settings within the PID were set to TRUE.

The system exhibits a reverse acting relationship between pump speed and engine outlet temperature: increasing pump speed lowers the temperature, while decreasing pump speed raises it. Because the pump cannot operate below zero output, offset and scaling adjustments were required to enable effective PID control.

Modifications to the PLC logic were implemented to support this constraint. With the control loop off, the engine outlet temperature stabilizes at approximately 273 K. Since the pump cannot run at negative speed to increase temperature, a baseline offset was added to the PID output using a math addition block before sending the signal through the Modbus write instruction. This establishes a minimum pump speed.

To ensure consistency between operating modes, manual inputs from the HMI are adjusted using a subtraction block before entering the PID block. Together, these changes allow both manual and automatic control to function properly.

AVENUES FOR CONTINUED WORK

There were many simulation components that were planned but not achieved with this project, leaving significant opportunities for future projects. First, the system is set up to allow for multiple independent copies (threads) of the simulation for the different lab stations. This is accomplished by having a separate port for each thread and assigning those ports to each lab station. Second, the simulation does not model branched flow, i.e. multiple heat exchangers in parallel with their valves, etc. Third, the simulation does not dynamically calculate convective heat transfer coefficient based upon the system flow rate. This appeared to be a reasonable assumption but could and should be verified in future iterations of the project. Finally, the system supports multiple ControlLoops which could allow for multiple-system interactions (i.e. a high temperature cooling loop interacting with and cooled by a saltwater cooling loop).

CONCLUSION

This project successfully developed a Raspberry Pi-based simulated industrial plant integrated with a SCADA HMI. The simulation models fluid power and thermodynamics using Modbus communication in Python, providing a realistic cooling loop driven by discrete thermal mass parcels, centrifugal pump curves, and iterative loss calculations.

The discrete thermal mass model proved to be a practical alternative to the lumped-parameter thermal network model, offering improved handling of transport delays and transient behavior without the computational cost of simultaneous root-finding. The Modbus server, running asynchronously via PyModbus, enabled reliable bidirectional communication between the Raspberry Pi and the PLC despite challenges involving adapter selection and register indexing conventions.

PLC integration through Connected Components Workbench and HMI development in Ignition demonstrated a complete automation stack from field simulation to supervisory control. The addition of a PID control loop with manual and automatic modes gave the system practical instructional value, adding the missing SCADA component to complete a full automation stack for the Maine Maritime Academy Automation Lab.

The platform is designed to be scalable and extensible. Future work could expand the simulation to include additional fluid systems, refine the convective heat transfer model to account for flow-dependent coefficients, and increase the number of fluid parcels to improve thermal accuracy.

ACKNOWLEDGEMENTS

The authors of this paper would like to thank our advisor, Professor Brendyn Sarnacki for the extensive subject-area knowledge he provided in the undertaking of this project. He was an invaluable resource for difficult conceptual discussions around the simulation design decisions that we made, and the project likely would not have found success without him. We are also immensely grateful for the project management guidance and generous support and wisdom provided by Professor Douglas Read.

REFERENCES

- Amazon.com: ESP32-S3 Ethernet Development Board, 10/100Mbps RJ45 ETH Port, 240MHz Dual Core Processor, Onboard Type-C Port & TF Card Slot, Support WiFi & BLE & Ethernet Communication, with POE Module : Electronics.
- Amazon.com: PNY 64GB Premier-X Class 10 U3 V30 microSDXC Flash Memory Card 5-Pack - 100MB/s, Class 10, U3, V30, A1, 4K UHD, Full HD, UHS-I, Micro SD : Electronics. Accessed 21 Apr. 2026.
- Ananev, Alexandr. Sanny32/OpenModScan. 23 Dec. 2022, C++. 16 Apr. 2026. GitHub, <https://github.com/sanny32/OpenModScan>.

“Asyncio — Asynchronous I/O.” Python Documentation, <https://docs.python.org/3/library/asyncio.html>. Accessed 21 Apr. 2026.

Automeris.Io: Computer Vision Assisted Data Extraction from Charts Using WebPlotDigitizer. <https://automeris.io/>. Accessed 30 Jan. 2026.

Brentq — SciPy v1.17.0 Manual. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.brentq.html>. Accessed 21 Apr. 2026.

Cooper Parlee. “Release v1.0.0 Presentation Build · CooperParlee/ReactorOnChip.” GitHub, <https://doi.org/10.5281/zenodo.19701554>. Accessed 22 Apr. 2026.

Jan Iversen. “PyModbus Documentation | Python Modbus Protocol Library.” PyModbus Documentation | Python Modbus Protocol Library, 2026, <https://www.pymodbus.com>.

“Modbus Register Addressing.” Continental Control Systems, LLC, <https://ctlsys.com/support/modbus-register-addressing/>. Accessed 21 Apr. 2026.

“Modbus Tool.” SourceForge, 22 June 2024, <https://sourceforge.net/projects/modbus-tool/>.

“Python Release Python 3.12.3.” Python.Org, <https://www.python.org/downloads/release/python-3123/>. Accessed 21 Apr. 2026.

“Raspberry Pi 5 8GB.” Camelcamelcamel.Com, <https://camelcamelcamel.com/product/B0CK2FCG1K>. Accessed 21 Apr. 2026.

“Threading — Thread-Based Parallelism.” Python Documentation, <https://docs.python.org/3/library/threading.html>. Accessed 21 Apr. 2026.

TP 40-240/2 A-G-A-BQQE-GX2 - 96411803 | Grundfos. <https://product-selection.grundfos.com/us/products/tp-tpe/tp-north-america/tp-40-2402-96411803?pumpsystemid=2871987636&tab=variant-quotation>. Accessed 21 Apr. 2026.

APPENDIX A1 REQUIRED ADDITIONAL PYTHON DEPENDENCIES

asyncio v4.0.0
colorama v0.4.6
contourpy v1.3.3
cyclor v0.12.1
et_xmlfile v2.0.0
fonttools v4.61.1
kiwisolver v1.4.9
matplotlib v3.10.8
numpy v2.4.0
openpyxl v3.1.5
packaging v25.0
pillow v12.1.0
pymodbus v3.11.4
pyparsing v3.3.1
PyQt6 v6.10.1
PyQt6-Qt6 v6.10.1
PyQt6_sip v13.10.3
pyqtgraph v0.14.0
python-dateutil v2.9.0.post0
scipy v1.17.0
six v1.17.0

APPENDIX A2
HOLDING/INPUT REGISTER LIST

Input Registers

Addr.	Information	Unit	Scale	Location
400	Reference node before pump. Should be about 0 kPa	kPa	100	Before pump
401	Pump discharge pressure	kPa	100	After pump
402	Cooler inlet pressure (the engine side)	kPa	100	Before hot HX
403	Cooler outlet pressure	kPa	100	After hot HX
404	Salt water cooler inlet pressure	kPa	100	Before the cold HX
405	Salt water cooler outlet pressure	kPa	100	Before cold HX
406	Engine cooler inlet temp	K	10	Before hot HX
407	Engine cooler outlet temp	K	10	After hot HX
408	Salt water cooler inlet temp	K	10	Before cold HX
409	Salt water cooler outlet temp	K	10	After cold HX
410	Pump speed	rpm	1	On pump

Holding Registers

Addr.	Information	Unit	Affects
300	Pump speed setpoint. Accepted range 0-1000	-	Pump
301	Engine load		